MoonWalker Weekly Meeting

11/13 - 11/19 Summary

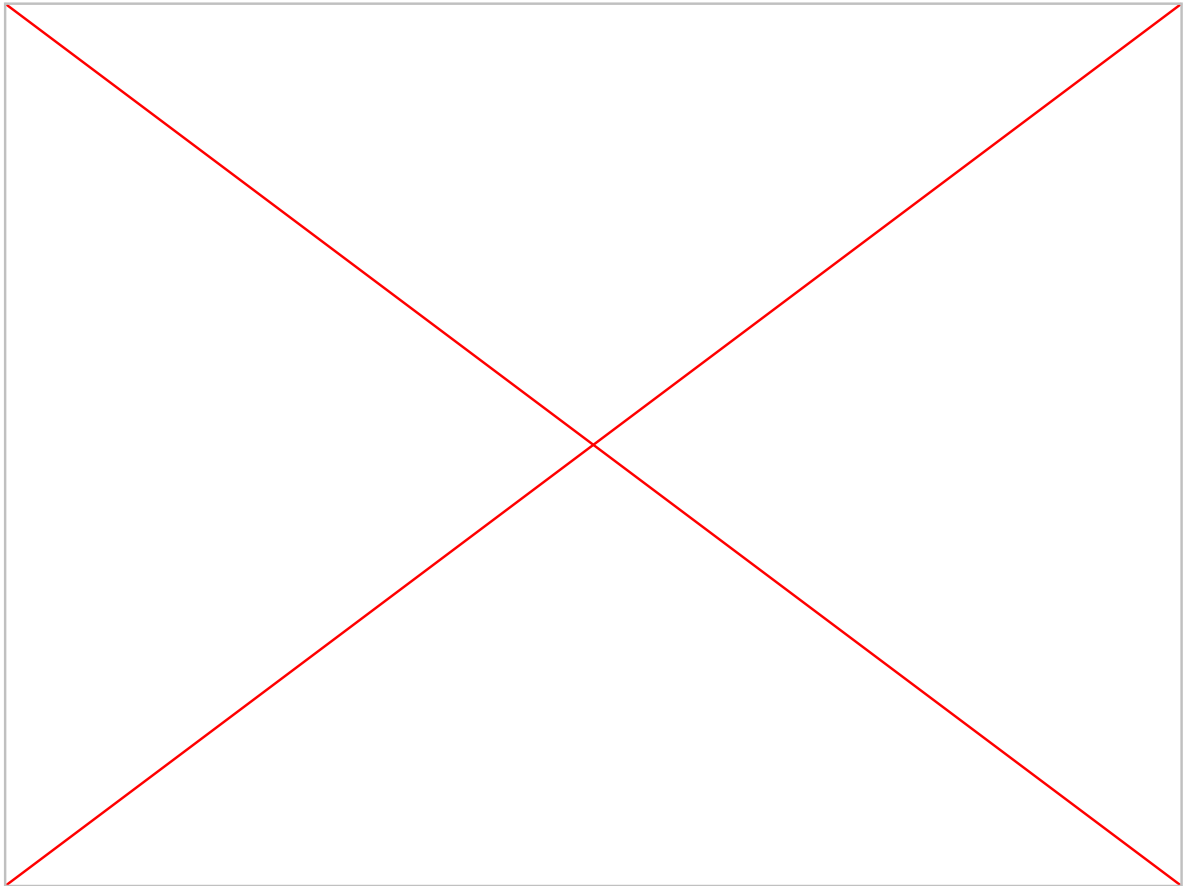
Elijah, Rowel, Martin

Progress Update

- Martin
  - While waiting to figure out “one-host” situation, tried out applying some ideas, namely removing redundant or mixed physics when colliding (related to the back-up idea).
  - On the subject of “one-host” situation, created a way in which when Unity Ghost Cadre is in a collision, it automatically changes scripts to start producing, making Unity the host controlling physics.
  - “Smoothen” Unity ghost cadre positional movement by decreasing the time it takes for Omniverse to send messages.
- Elijah
  - Tested collision with Martin’s fix
  - Research on Omniverse physics parameters to pass
  - Started implementing physics parameter extractor
- Rowel
  - Began Testing with Martin’s Implementation Plan and continued research and began implementation of hosting platform and passing physics parameters in JSON(Shared)
  - Implemented auto-clearing Kafka topics to minimize lag and glitching during testing
  - Began writing latency tests to see the difference between production and consumption of messages

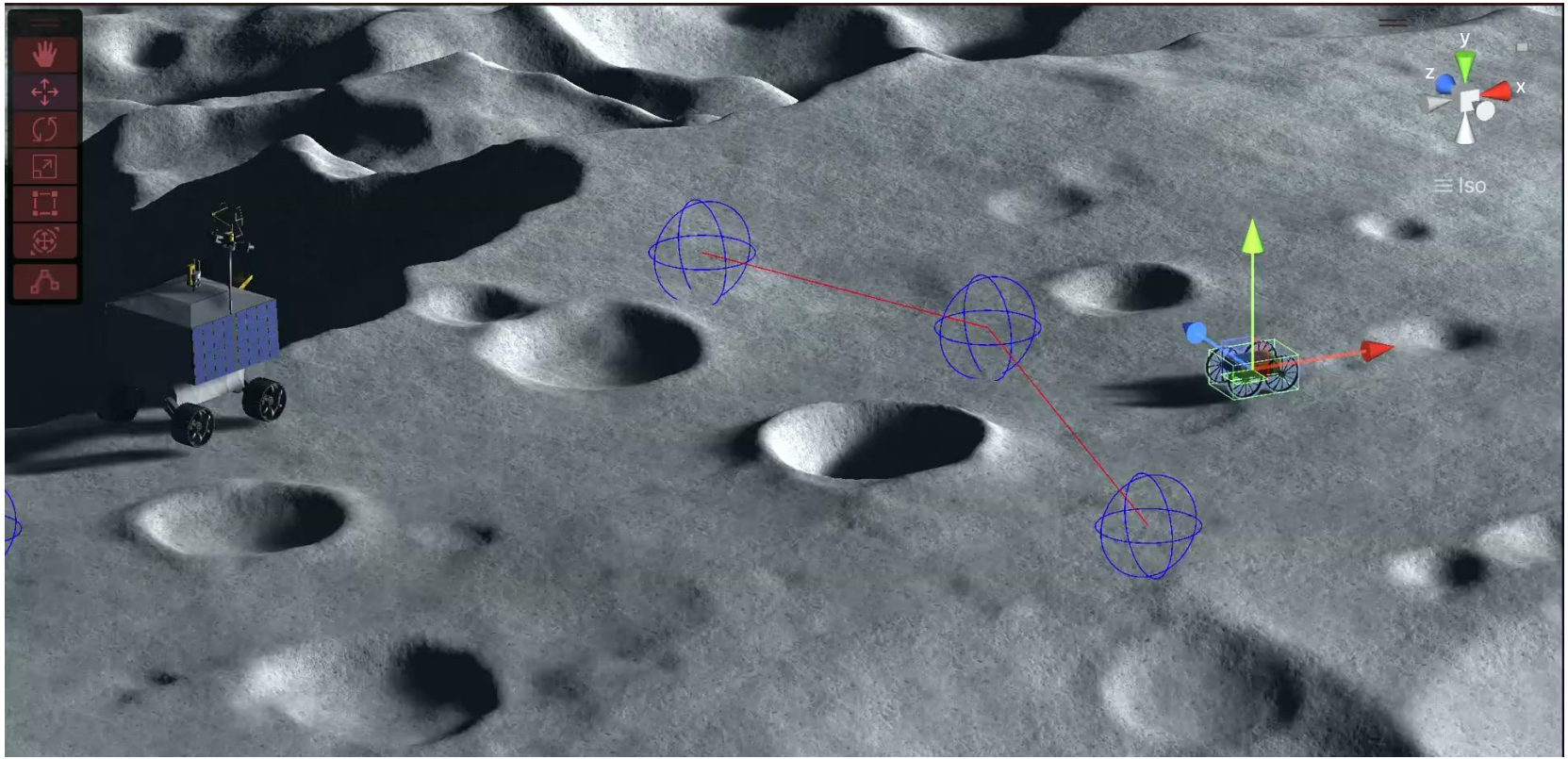
Collision Test I - Omniverse

Collision Test I - Unity



Collision Test II - Omniverse

Collision Test II - Unity



Collision Tests - Analysis

- After reducing mixed physics, Unity Rigidbody “is Kinematic” on the ghost/proxy objects.
  - Collisions became noticeably more similar between Omniverse and Unity
- With that change, objects showed similar time in the air (hang-time) in both sims during collisions
- On the Unity side, the Cadre position coordinates are correct, but the rotation/orientation isn’t perfect during collisions.
  - Rotations were fairly similar across engines post-impact
  - Possible issue might be related to Kafka messaging. We know that rovers tend to act strange when reading old or empty messages.
  - We find that Unity produces close to thousands of messages while Omniverse produces only hundreds - potential path to look into.
- Results have been repeatable during current testing

Physics Parameters (Omniverse)

- Scene-level Physics Parameters
  - Gravity
  - Units: linearUnits (meters, centimeters) or angularUnits (radians, degrees)
- Rigid Body
  - Mass
  - Inertia: inertiaTensor
  - Center of mass: [x, y, z] in local coordinates
  - Kinematic flag: isKinematic (boolean), might be important for hosting_flag implementation
- Collider parameters:
  - Shape type: shapeType (box, sphere, capsule, cylinder, convexMesh, triangleMesh)

Example JSON file (mock-up)

```
{
    "metadata":{
        "source": "omniverse",
        "version": "1.0"
    },

    "scene": {
        "units": {
            "linear": "meters",
            "angular": "radians",
            "meters_per_unit": 1.0,
            "up_axis": "z"
        },
        "gravity": {
            "direction": [0.0, 0.0, -1.0],
            "magnitude": 9.81
        }
    },
```

scene-level physics

```
"bodies": [
{
    "name": "base_link",
    "path": "/World/~",
    "mass": 20.0,
    "inertia":{
        "type": "full_matrix",
        "values": [
        [0.8, 0.0, 0.0],
        [0.0, 0.9, 0.0],
        [0.0, 0.0, 1.0]
        ]
    },
    "center_of_mass": [0.0, 0.15, 0.0],
    "is_kinematic": false,
    "collision_shapes": [
    {
        "shape_type": "box",
        "local_offset": {
            "position": [0.0, 0.15, 0.0],
            "rotation_quat": [1.0, 0.0, 0.0, 0.0]
        }
    }
    ]
},
```

rigid-body and collision parameters

Example JSON file (mock-up)

- Note: Most of attribute extraction functions are: Get{name}Attr().Get()

1. Gravity:
   a. GetGravityDirectionAttr().Get()
   b. GetGravityMagnitudeAttr().Get()
2. linearUnits:
   a. GetStageMetersPerUnit(stage)
3. Mass
   a. GetMassAttr().Get()
4. Inertia:
   a. GetInertiaAttr().Get()
5. Kinematic Flag:
   a. GetKinematicEnabledAttr().Get()
6. Collision Shape Type:
   a. GetTypeName()

```
{
    "name": "wheel1",
    "path": "/World/~/wheel1",
    "mass": 2.0,
    "inertia": {
        "type": "diagonal",
        "values": [0.02, 0.04, 0.02]
    },
    "center_of_mass": [0.0, 0.0, 0.0],
    "is_kinematic": false,
    "collision_shapes":[
    {
        "shape_type": "cylinder",
        "local_offset":{
            "position": [0.0, 0.0, 0.0],
            "rotation_quat": [1.0, 0.0, 0.0, 0.0]
        }
    }
    ]
},
...
```

rigid-body and collision parameters

Code

```python
def get_units_and_gravity(stage):
    # --- Units ---
    meters_per_unit = UsdGeom.GetStageMetersPerUnit(stage)
    up_axis = UsdGeom.GetStageUpAxis(stage)  # "y" or "z"

    # --- Gravity: find first UsdPhysics.Scene in the stage ---
    gravity_dir = Gf.Vec3f(0.0, 0.0, -1.0)
    gravity_mag = 9.81

    physics_scene = None
    for prim in stage.Traverse():
        scene_api = UsdPhysics.Scene(prim)
        if scene_api and scene_api.GetPrim().IsValid():
            physics_scene = scene_api
            break

    if physics_scene is not None:
        dir_attr = physics_scene.GetGravityDirectionAttr()
        mag_attr = physics_scene.GetGravityMagnitudeAttr()
        if dir_attr.IsAuthored():
            gravity_dir = dir_attr.Get()
        if mag_attr.IsAuthored():
            gravity_mag = mag_attr.Get()

    return {
        "units": {
            "linear": "meters",           # assuming SI; change if needed
            "angular": "radians",         # Isaac uses radians
            "meters_per_unit": float(meters_per_unit),
            "up_axis": up_axis
        },
        "gravity": {
            "direction": [float(gravity_dir[0]),
                          float(gravity_dir[1]),
                          float(gravity_dir[2])],
            "magnitude": float(gravity_mag)
        }
    }
```

```python
def get_rigid_body_physics(stage, prim_path):
    prim = stage.GetPrimAtPath(prim_path)
    if not prim or not prim.IsValid():
        raise RuntimeError(f"Prim not found or invalid: {prim_path}")

    mass_api = UsdPhysics.MassAPI(prim)
    rb_api = UsdPhysics.RigidBodyAPI(prim)
    physx_rb_api = PhysxSchema.PhysxRigidBodyAPI(prim)

    # Mass
    mass = None
    if mass_api and mass_api.GetPrim().IsValid():
        mass = mass_api.GetMassAttr().Get()

    # Center of mass
    com = None
    if mass_api and mass_api.GetPrim().IsValid():
        com = mass_api.GetCenterOfMassAttr().Get()

    # Inertia:
    # Prefer diagonal inertia if present; otherwise try full inertia tensor
    inertia_type = None
    inertia_values = None
    if mass_api and mass_api.GetPrim().IsValid():
        diag = mass_api.GetDiagonalInertiaAttr().Get()
        if diag is not None:
            inertia_type = "diagonal"
            inertia_values = [float(diag[0]), float(diag[1]), float(diag[2])]
        else:
            full = mass_api.GetInertiaAttr().Get()
            if full is not None:
                # full is 9 elements (row-major); convert to 3x3 for readability
                inertia_type = "full_matrix"
                inertia_values = [
                    [float(full[0]), float(full[1]), float(full[2])],
                    [float(full[3]), float(full[4]), float(full[5])],
                    [float(full[6]), float(full[7]), float(full[8])]
                ]

    # Kinematic flag (PhysX-specific)
    is_kinematic = False
    if physx_rb_api and physx_rb_api.GetPrim().IsValid():
        val = physx_rb_api.GetKinematicEnabledAttr().Get()
        if val is not None:
            is_kinematic = bool(val)
```

Action Plan

- Rowel
  - Continue testing to verify the current implementation provides more identical collision (Shared)
  - Finish and use Latency tests
- Elijah
  - Continue testing to verify the current implementation provides more identical collision (Shared)
  - Finish scripts: physics parameters extraction script, Omniverse cadreA consumer script
- Martin
  - Continue testing to verify the current implementation provides more identical collision (Shared)
  - Look into difference in messages in Kafka between Unity and Omniverse
  - Regarding “one-hosting”, move related scripts from Unity to Omniverse instead to have Omniverse be the host.